

Trabajo Práctico Integrador – Programación II

Integrantes y roles

Franco Herrera (Service y DAOs)

Damian Tristant (Models)

Ximena Sosa (Main - AppMenu)

Elección del Dominio

Elegimos las entidades Paciente – HistoriaClinica. También agregamos el enum GrupoSanguineo para parametrizar los grupos sanguíneos de los pacientes. En nuestro dominio, Paciente e HistoriaClinica tienen una relación de composición, ya que un paciente “tiene” una historia clínica. La relación es 1:1 (un paciente tiene una historia clínica y una historia clínica tiene un paciente).

Arquitectura por capas

Nuestro proyecto fue desarrollado siguiendo una arquitectura en capas, para una clara separación de responsabilidades. La misma facilita el mantenimiento, la extensibilidad y el testing de cada componente. Las capas implementadas son:

1. Capa de Presentación: main – AppMenu
2. Capa de Servicios: service
3. Capa de Acceso a Datos: dao
4. Capa de Entidades/Modelos: models – enums
5. Capa de configuración: config

Capa de Presentación

La Capa de Presentación tiene la responsabilidad principal de interactuar directamente con el usuario. Es la interfaz visible de la aplicación. Está compuesta por **main**, que es el punto de entrada de la aplicación. **main** está encargada de inicializar y ejecutar **AppMenu**. **AppMenu** contiene la lógica del menú de consola, presenta las opciones al usuario y recopila las entradas. Además, creamos los métodos auxiliares `getString()`, `getInt()` y `getDate()` para facilitar el manejo robusto de entradas inválidas. Estos métodos encapsulan la lógica para evitar *NumberFormatException*, en el caso de `getInt()` y *DateTimeParseException* en el caso de `getDate()`.

Capa de Servicios

La capa de Servicios tiene la responsabilidad de contener y ejecutar la lógica de negocio de la aplicación. Orquesta las operaciones y gestiona las transacciones de base

de datos. Está compuesta por la interfaz **IGenericService<T>** que define las operaciones CRUD genéricas. Además, las interfaces **IHistoriaClinicaService** y **IPacienteService** extienden la interfaz genérica y añaden métodos de negocio específicos, como *buscarPorDni* o *buscarPorNroHistoria*. Por último, **HistoriaClinicaService** y **PacienteService** implementan las interfaces, contienen lógica de negocio, validaciones y la gestión de las transacciones.

Capa de Acceso a Datos (DAO)

La Capa de Acceso a Datos tiene la responsabilidad de abstraer y encapsular la lógica de comunicación y persistencia con la base de datos. Es la única capa de la aplicación que sabe cómo interactuar con la base de datos. Está compuesta por **DataBaseConnection**, en config, que funciona como clase auxiliar para establecer conexiones a la base de datos, leyendo la configuración externa (**database.properties**). Además, contiene la interfaz genérica **IGenericDAO<T>** que define los contratos genéricos para las operaciones CRUD. De igual manera que en la Capa de Servicios, optamos por utilizar las interfaces específicas **IPacienteDAO** e **IHistoriaClinicaDAO** para extender a la interfaz genérica y añadir métodos específicos. Por último, **PacienteDAO** e **HistoriaClinicaDAO** se encargan de implementar las interfaces y contener el código JDBC para ejecutar las sentencias SQL directamente contra la base de datos.

Capa de Entidades/Modelos

La Capa de Entidades/Modelos representa las estructuras de datos y los objetos del dominio de la aplicación. Está compuesta por la clase abstracta **Base**, que encapsula los atributos comunes id y eliminado, tal como lo muestra el video que nos fue sugerido en la consigna. También, tiene la clase **Paciente**, que representa la entidad principal ("A") del dominio y la clase **HistoriaClinica**, que representa la entidad "B". Por último, creamos el enum **GrupoSanguineo** para manejar los datos fijos.

Capa de Configuración

La Capa de Configuración contiene **DatabaseConnection** que obtiene las propiedades para conectarse a la base de datos y un método estático que retorna la conexión con la misma.

Persistencia de Datos

En nuestro proyecto, la persistencia se implementa utilizando JDBC (Java Database Connectivity) para interactuar con una base de datos MySQL Server. La base de datos **hospital** fue diseñada siguiendo un modelo relacional que refleja las entidades principales del sistema y la relación de composición entre ellas. Se compone de las siguientes tablas:

- grupo_sanguineo (catálogo): Normaliza los grupos sanguíneos, asegurando coherencia y facilitando su gestión. Está compuesta por un identificador único y la descripción del grupo sanguíneo.
- historia_clinica (entidad B): Almacena los registros médicos de un paciente. Está compuesta por un identificador único, eliminado, para la baja lógica, nro_historia, id_grupo_sanguineo, que es la FK a grupo_sanguineo, antecedentes, medicacion_actual y observaciones, que son campos de texto para información detallada.
- paciente (entidad A): Almacena los pacientes del hospital. Está compuesta por un identificador único, eliminado, para la baja lógica, nombre y apellido, dni, fecha_nacimiento y id_historia, que es la FK a historia_clinica.

Si bien la consigna sugería “clave foránea única en la tabla de B”, optamos por crear la clave foránea en paciente, ya que consideramos que para cumplir con la relación de composición, es necesario que un **Paciente** contenga una **HistoriaClinica**.

El manejo de transacciones es fundamental para asegurar la integridad y consistencia de los datos, especialmente en operaciones que involucran múltiples llamadas a la base de datos, como insertar un paciente y su historia clínica. En nuestro proyecto, las transacciones se gestionan en la Capa de Servicios. Por ejemplo, en el método *insertar(Paciente paciente)*:

```

29 // METODO PARA INSERTAR PACIENTE
30 @Override
31 public void insertar(Paciente paciente) throws SQLException {
32     // VALIDACIONES
33     Objects.requireNonNull(paciente, "El Paciente no puede ser nulo.");
34     Objects.requireNonNull(paciente.getNombre(), "El nombre del paciente es obligatorio.");
35     Objects.requireNonNull(paciente.getApellido(), "El apellido del paciente es obligatorio.");
36     Objects.requireNonNull(paciente.getDni(), "El DNI del paciente es obligatorio.");
37     Objects.requireNonNull(paciente.getFechaNacimiento(), "La fecha de nacimiento es obligatoria.");
38     Objects.requireNonNull(paciente.getHistoriaClinica(), "La Historia Clinica asociada es obligatoria para el paciente.");
39
40     // VALIDACIONES PARA LA HISTORIA CLINICA
41     Objects.requireNonNull(paciente.getHistoriaClinica().getNroHistoria(), "El número de historia clinica es obligatorio.");
42     Objects.requireNonNull(paciente.getHistoriaClinica().getGrupoSanguineo(), "El grupo sanguineo de la HC es obligatorio.");
43
44     Connection con = null;
45     try {
46         con = DatabaseConnection.getConnection();
47         con.setAutoCommit(false);
48
49         // VALIDO QUE EL DNI NO EXISTA
50         Paciente pacienteExistente = pacienteDao.buscarPorDni(con, paciente.getDni());
51         if (pacienteExistente != null && !pacienteExistente.isEliminado()) {
52             throw new SQLException("Ya existe un Paciente con el DNI " + paciente.getDni());
53         }
54
55         // VALIDO QUE EL NRO DE LA HISTORIA CLINICA NO EXISTA
56         HistoriaClinica hcExistente = historiaClinicaDao.buscarPorNroHistoria(con, paciente.getHistoriaClinica().getNroHistoria());
57         if (hcExistente != null && !hcExistente.isEliminado()) {
58             throw new SQLException("Ya existe una Historia Clinica con el número " + paciente.getHistoriaClinica().getNroHistoria());
59         }
60
61         historiaClinicaDao.crear(con, paciente.getHistoriaClinica());
62         pacienteDao.crear(con, paciente);
63
64         con.commit();
65
66     } catch (SQLException e) {
67         if (con != null) {
68             try {
69                 con.rollback();
70             } catch (SQLException ex) {
71                 System.err.println("Error al realizar rollback: " + ex.getMessage());
72             }
73         }
74         throw e;
75     }
76 }

```

Se puede observar que iniciamos la transacción con `con.setAutoCommit(false)` justo antes de hacer las validaciones, dentro del bloque del `try`. Una vez creados paciente e historia clínica, confirmamos los cambios con `con.commit()`. En el bloque `catch`, al cuál se va a acceder en caso de que el programa falle, deshacemos todos los cambios que se hicieron previamente al fallo, utilizando `con.rollback()`.

```
} finally {
    if (con != null) {
        try {
            con.setAutoCommit(true);
            con.close();
        } catch (SQLException e) {
            System.err.println("Error al cerrar la conexión: " + e.getMessage());
        }
    }
}
```

Por último, en el bloque `finally`, se restablece el modo de auto commit a `true`.

Validaciones y reglas de negocio

La lógica de negocio de nuestra aplicación, junto con las validaciones de los datos, reside principalmente en la Capa de Servicios. Esta capa es responsable de asegurar que los datos sean coherentes y cumplan con las reglas definidas antes de interactuar con la base de datos.

Al momento de insertar un paciente, por ejemplo, validamos que el DNI ingresado no exista ya en la base de datos. Lo mismo hacemos para el número de historia clínica. Por otro lado, cuando actualizamos un paciente existente, lo primero que hacemos es buscar al paciente por ID y luego validamos que exista. Después, procedemos a validar los nuevos valores que se le asignan a su DNI y al número de su historia clínica. En el caso de los campos que son obligatorios, utilizamos la clase auxiliar **Objects**, específicamente el método `requireNonNull`. Este método verifica si un objeto de referencia que se le pasa como argumento es `null`. Si el objeto es `null`, lanza una `NullPointerException`. Su uso fue sugerido por la IA, como consecuencia de una búsqueda para facilitar el manejo de los campos que definimos en nuestra base de datos como no nulos.

Pruebas

Después de crear las tablas, ejecutamos el script de inserción de datos, por lo que ya contamos con 999 registros en nuestra base de datos.

	id	eliminado	nombre	apellido	dni	fecha_nacimiento	id_historia
▶	1	0	María	Gomez	30000001	1965-11-07	1
	2	0	Carlos	Blanco	30000002	2006-07-22	2
	3	0	Ana	Perez	30000003	2003-05-31	3
	4	0	Franco	Fernandez	30000004	1989-06-30	4
	5	0	Mateo	Martinez	30000005	1991-04-25	5
	6	0	Agustina	Diaz	30000006	1980-03-11	6
	7	0	Nicolas	Herrera	30000007	1981-01-28	7
	8	0	Sofia	Gonzalez	30000008	1956-12-02	8
	9	0	Roberto	Lopez	30000009	1957-05-25	9
	10	0	Luis	Romero	30000010	1970-04-15	10
	11	0	Juan	Rodriguez	30000011	1971-05-07	11
	12	0	María	Gomez	30000012	1999-12-09	12
	13	0	Carlos	Blanco	30000013	1953-11-04	13
	14	0	Ana	Perez	30000014	1947-05-20	14
	15	0	Franco	Fernandez	30000015	1991-05-26	15
	16	0	Mateo	Martinez	30000016	1983-01-18	16
	17	0	Agustina	Diaz	30000017	1995-02-07	17
	18	0	Nicolas	Herrera	30000018	1956-07-08	18
	19	0	Sofia	Gonzalez	30000019	1975-04-06	19
	20	0	Roberto	Lopez	30000020	1998-11-11	20
	21	0	Luis	Romero	30000021	1998-09-03	21
	22	0	Juan	Rodriguez	30000022	1988-11-20	22

Listar alumnos

```
run:
--- Sistema de gestión de Pacientes ---
1. Ingresar nuevo Paciente
2. Buscar Paciente por ID
3. Listar todos los Pacientes
4. Actualizar datos de Paciente
5. Eliminar Paciente
6. Buscar Paciente por DNI
0. Salir
Ingrese una opción: |
```

```
Ingrese una opción: 3
--- Listado de Pacientes ---
Paciente[nombre= María, apellido= Gomez, dni= 30000001, fecha_nacimiento= 1965-11-07, historia_clinica= HC001]
Paciente[nombre= Carlos, apellido= Blanco, dni= 30000002, fecha_nacimiento= 2006-07-22, historia_clinica= HC002]
Paciente[nombre= Ana, apellido= Perez, dni= 30000003, fecha_nacimiento= 2003-05-31, historia_clinica= HC003]
Paciente[nombre= Franco, apellido= Fernandez, dni= 30000004, fecha_nacimiento= 1989-06-30, historia_clinica= HC004]
Paciente[nombre= Mateo, apellido= Martinez, dni= 30000005, fecha_nacimiento= 1991-04-25, historia_clinica= HC005]
Paciente[nombre= Agustina, apellido= Diaz, dni= 30000006, fecha_nacimiento= 1980-03-11, historia_clinica= HC006]
Paciente[nombre= Nicolas, apellido= Herrera, dni= 30000007, fecha_nacimiento= 1981-01-28, historia_clinica= HC007]
Paciente[nombre= Sofia, apellido= Gonzalez, dni= 30000008, fecha_nacimiento= 1956-12-02, historia_clinica= HC008]
Paciente[nombre= Roberto, apellido= Lopez, dni= 30000009, fecha_nacimiento= 1957-05-25, historia_clinica= HC009]
Paciente[nombre= Luis, apellido= Romero, dni= 30000010, fecha_nacimiento= 1970-04-15, historia_clinica= HC010]
Paciente[nombre= Juan, apellido= Rodriguez, dni= 30000011, fecha_nacimiento= 1971-05-07, historia_clinica= HC011]
Paciente[nombre= María, apellido= Gomez, dni= 30000012, fecha_nacimiento= 1999-12-09, historia_clinica= HC012]
Paciente[nombre= Carlos, apellido= Blanco, dni= 30000013, fecha_nacimiento= 1953-11-04, historia_clinica= HC013]
Paciente[nombre= Ana, apellido= Perez, dni= 30000014, fecha_nacimiento= 1947-05-20, historia_clinica= HC014]
Paciente[nombre= Franco, apellido= Fernandez, dni= 30000015, fecha_nacimiento= 1991-05-26, historia_clinica= HC015]
Paciente[nombre= Mateo, apellido= Martinez, dni= 30000016, fecha_nacimiento= 1983-01-18, historia_clinica= HC016]
Paciente[nombre= Agustina, apellido= Diaz, dni= 30000017, fecha_nacimiento= 1995-02-07, historia_clinica= HC017]
Paciente[nombre= Nicolas, apellido= Herrera, dni= 30000018, fecha_nacimiento= 1956-07-08, historia_clinica= HC018]
Paciente[nombre= Sofia, apellido= Gonzalez, dni= 30000019, fecha_nacimiento= 1975-04-06, historia_clinica= HC019]
```

Buscar paciente por DNI

```
--- Sistema de gesti3n de Pacientes ---
1. Ingresar nuevo Paciente
2. Buscar Paciente por ID
3. Listar todos los Pacientes
4. Actualizar datos de Paciente
5. Eliminar Paciente
6. Buscar Paciente por DNI
0. Salir
Ingrese una opci3n: 6
--- Buscar Paciente por DNI ---
Ingrese DNI del paciente:
```

```
Ingrese DNI del paciente:
30000016

--- Datos del Paciente ---
Paciente(nombre= Mateo, apellido= Martinez, dni= 30000016, fecha_nacimiento= 1983-01-18, historia_clinica= HC016)

--- Historia Cl3nica ---
HistoriaClinica(numero= HC016, grupo_sanguineo= A_POSITIVO, antecedentes= Antecedente gen3rico 16, medicacion= Medicaci3n gen3rica 16, observaciones= Observaci3n gen3rica 16)
```

Actualizar paciente

```
--- Sistema de gesti3n de Pacientes ---
1. Ingresar nuevo Paciente
2. Buscar Paciente por ID
3. Listar todos los Pacientes
4. Actualizar datos de Paciente
5. Eliminar Paciente
6. Buscar Paciente por DNI
0. Salir
Ingrese una opci3n: 4
--- Actualizar Datos de Paciente ---
Ingrese el ID del Paciente a actualizar: 56
```

```

--- Datos del Paciente ---
Nombre actual: María
Nuevo nombre:
Juana
Apellido actual: Gomez
Nuevo apellido:
Gomes
DNI actual: 30000056
Nuevo DNI:
40000000
Fecha Nacimiento actual: 2004-01-21
Nueva Fecha de Nacimiento (YYYY-MM-DD): 2004-01-21
Nro. actual: HC056
Nuevo Nro de Historia:
HC056
Grupo Sanguineo actual: A+
Nuevo Grupo sanguineo (ej: A+, O-):
O-
Antecedentes actuales: Antecedente genérico 56
Nuevos Antecedentes:
Alergia
Medicación actual: Medicación genérica 56
Nueva Medicación:
N/A
Observaciones actuales: Observación genérica 56
Nuevas Observaciones:
N/A
Paciente actualizado correctamente.

```

Busco el paciente actualizado

```

--- Sistema de gestión de Pacientes ---
1. Ingresar nuevo Paciente
2. Buscar Paciente por ID
3. Listar todos los Pacientes
4. Actualizar datos de Paciente
5. Eliminar Paciente
6. Buscar Paciente por DNI
0. Salir
Ingrese una opción: 6
--- Buscar Paciente por DNI ---
Ingrese DNI del paciente:
40000000

--- Datos del Paciente ---
Paciente(nombre= JUANA, apellido= GOMES, dni= 40000000, fecha_nacimiento= 2004-01-21, historia_clinica= HC056)

--- Historia Clínica ---
HistoriaClinica(numero= HC056, grupo_sanguineo= O_NEGATIVO, antecedentes= ALERGIA, medicacion= N/A, observaciones= N/A)

```

Eliminar paciente

```
--- Sistema de gesti3n de Pacientes ---
1. Ingresar nuevo Paciente
2. Buscar Paciente por ID
3. Listar todos los Pacientes
4. Actualizar datos de Paciente
5. Eliminar Paciente
6. Buscar Paciente por DNI
0. Salir
Ingrese una opci3n: 5
--- Eliminar Paciente ---
Ingrese el ID del Paciente a eliminar: 56
El Paciente con ID 56 ha sido eliminado.
```

Busco al paciente eliminado

```
--- Sistema de gesti3n de Pacientes ---
1. Ingresar nuevo Paciente
2. Buscar Paciente por ID
3. Listar todos los Pacientes
4. Actualizar datos de Paciente
5. Eliminar Paciente
6. Buscar Paciente por DNI
0. Salir
Ingrese una opci3n: 2
--- Buscar Paciente ---
Ingrese el ID del Paciente: 56
El paciente con ID 56 no se encuentra.
```

Insertar paciente


```

--- Sistema de gesti3n de Pacientes ---
1. Ingresar nuevo Paciente
2. Buscar Paciente por ID
3. Listar todos los Pacientes
4. Actualizar datos de Paciente
5. Eliminar Paciente
6. Buscar Paciente por DNI
0. Salir
Ingrese una opci3n: 1
--- Crear nuevo Paciente ---
Nombre:
Franco
Apellido:
Herrera
DNI:
42680156
Fecha de Nacimiento (YYYY-MM-DD): 2000-05-09

--- Datos de Historia Clinica ---
N3mero de Historia Cl3nica:
HC1000
Grupo sanguineo (ej: A+, O-):
A+
ANTECEDENTES:
N/A
MEDICACI3N ACTUAL:
N/A
OBSERVACIONES:
N/A
Paciente creado exitosamente.

```

Busco el paciente ingresado

```

--- Sistema de gesti3n de Pacientes ---
1. Ingresar nuevo Paciente
2. Buscar Paciente por ID
3. Listar todos los Pacientes
4. Actualizar datos de Paciente
5. Eliminar Paciente
6. Buscar Paciente por DNI
0. Salir
Ingrese una opci3n: 6
--- Buscar Paciente por DNI ---
Ingrese DNI del paciente:
42680156

--- Datos del Paciente ---
Paciente{nombre= FRANCO, apellido= HERRERA, dni= 42680156, fecha_nacimiento= 2000-05-09, historia_clinica= HC1000}

--- Historia Cl3nica ---
HistoriaClinica{numero= HC1000, grupo_sanguineo= A_POSITIVO, antecedentes= N/A, medicacion= N/A, observaciones= N/A}

```

Conclusiones y mejoras futuras

El desarrollo del Trabajo Final Integrador nos permiti3 aplicar y consolidar de manera pr3ctica los principios de la Programaci3n Orientada a Objetos y el dise1o de software por capas. Aprendimos a conectar nuestra aplicaci3n en Java con una base de datos

MySQL y poder hacer operaciones CRUD. Investigamos acerca de la arquitectura en capas y la aplicamos a través del uso de Models, Services y DAOs. Aplicamos la relación de composición al proyecto, trabajamos de manera colaborativa, debatimos la relación 1:1 unidireccional y acordamos el dominio a utilizar.

Como mejoras futuras, consideramos como buena medida integrar una interfaz de usuario gráfica, ya que actualmente la interfaz es de consola. También, a lo largo del proyecto, utilizamos SQLException, pero podríamos implementar una jerarquía de excepciones de negocio más específica. También, al momento de listar pacientes, se podría implementar paginación. Por último, consideramos que se podrían añadir más métodos de búsqueda avanzada.

Herramientas utilizadas

T3.chat (Gemini 2.5 Flash y Claude 4.5 Sonnet)

Videos proporcionados en la sección Trabajo Práctico Integrador del Campus